SyPy
Security & Privacy
Research Group

Lecture 3 - Basics of ML
Part-1

Prof. Ankit Gangwal
Assistant Professor, IIIT-H, India
email: *gangwal@iiit.ac.in*
Web: *CiaoAnkit.github.io*

Recap

Hypothesis Testing

- Null hypothesis
  - Set of events
  - Status quo is maintained
- Alternate hypothesis
  - The claim to be made
  - Also a set of events
- These two are mutually disjoint and exhaustive

Recap

- Types of data
- Statistical tests
  - T-test: To compare the means of groups
- Significance level (α)
- p-value
  - P[(observing the given data) | (null hypothesis is true)]

Feedback we received

SyPy
Security & Privacy
Research Group

can we get some resources on where to study for quiz? also how much PNS will be needed in this course :(?

Pay attention to the lectures, and use the slides.

There will be resources given in the slides as well (blogs, videos, etc.)

As for PNS…
You're in luck! Not much required other than basic concepts of probabilities and random variables :)

Feedback we received

SyPy
Security & Privacy
Research Group

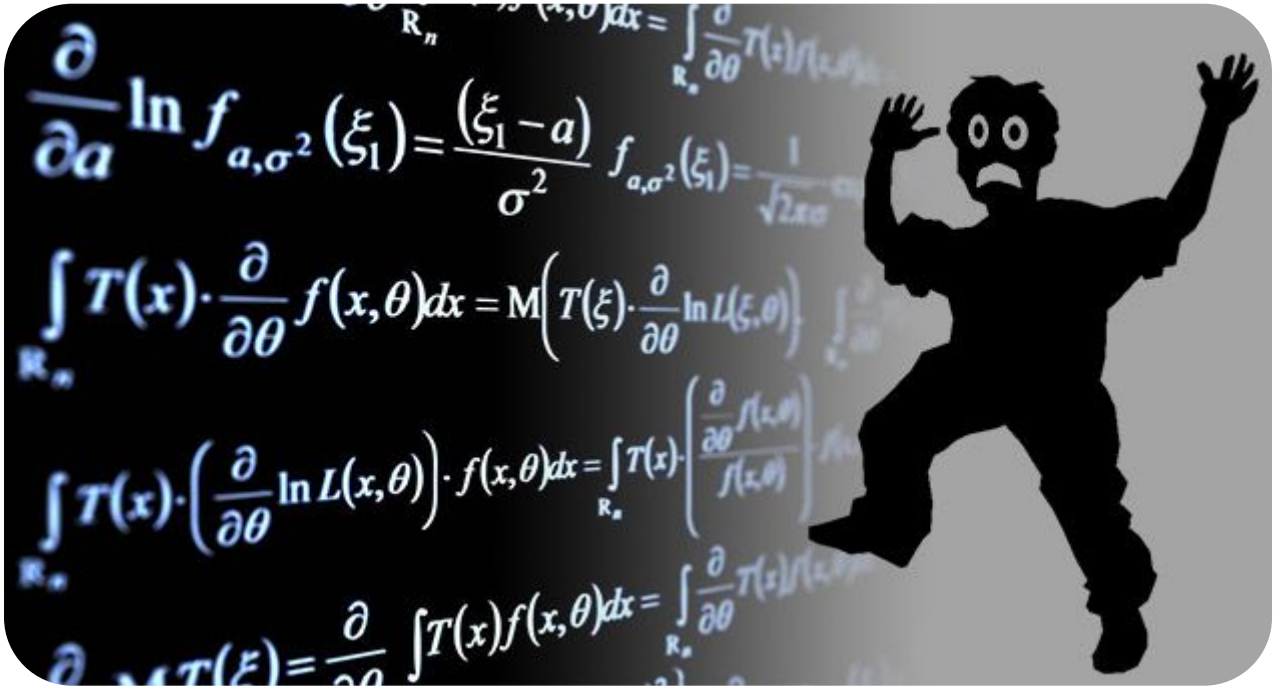
Giving fully end to end complete example of using all the concepts together and steps/process to follow to analyse it and finally show the calculations and get the answer and analyse it, it would be beneficial, felt most of the examples are specific to each small concepts, examples connecting all the related concepts and starting from a real life application type usecase and explained end to end from scratch would be really useful

First, I appreciate the informative response :)

We will try our best to do something similar for most of the concepts, but sometimes we may not be able to cover real-life use cases from end-to-end in complete depth (we will provide high-level intuition though).

If you want certain topics to be revisited with more examples, send us a mail and we'll try to cover them in one of the tutorials

A Word of Advice



Be not afraid of math!

It is not a dark art or some otherworldly abomination

Just try to have fun with it :)

A Word of Caution

SyPy
Security & Privacy
Research Group

Beware:
No spoilers!

Table of Contents

SyPy
Security & Privacy
Research Group

- What is ML?
- Why is ML?
- Basic Workflow of ML
- Basic Terminology in ML
- Tasks involved in Supervised Learning
  - Regression
  - Classification
- Perceptron
- Activation Functions
- Multi-layer perceptron

Table of Contents

SyPy Security & Privacy Research Group

- Learning parameters of a neural network
- Loss Functions in neural networks
  - Mean square error
  - Cross entropy
- Backpropagation, mini-batch training, SGD,
- Evaluating ML models
- Overfitting vs Underfitting
- Early Stopping
- A Very Short Introduction to Information Theory
- Cross Entropy/KL Divergence

SyPy Security & Privacy Research Group

Before we begin, let us clarify what we're going to focus on

The field of "Artificial Intelligence" is vast

SyPy Security & Privacy Research Group

A History in Brief:
The Origins of Artificial Intelligence

The field of AI research was founded at a workshop at Dartmouth College in 1956.

McCulloch and Pitts design for "artificial neurons" in 1943,

Researchers in the 1960s and the 1970s were convinced that their methods would eventually succeed in creating a machine with general intelligence and considered this the goal of their field.[364] In 1965 Herbert Simon predicted,

precise logic.[87][378] But the most important development was the revival of "connectionism", including neural network research, by Geoffrey Hinton and others.[379] In 1990, Yann LeCun successfully showed that convolutional

[Credits to Wikipedia]

SyPy
Security & Privacy
Research Group

But what is “Machine Learning”?

SyPy
Security & Privacy
Research Group

But what is “Machine Learning”?

From Wikipedia:
Machine learning (**ML**) is a field of study in artificial intelligence concerned with the development and study of statistical algorithms that can learn from data and generalize to unseen data, and thus perform tasks without explicit instructions.

SyPy
Security & Privacy
Research Group

But what is “Machine Learning”?

From Wikipedia:
Machine learning (**ML**) is a field of study in artificial intelligence concerned with the development and study of statistical algorithms that can learn from data and generalize to unseen data, and thus perform tasks without explicit instructions.

Nowadays, the land of buzzwords

Basics of Machine Learning

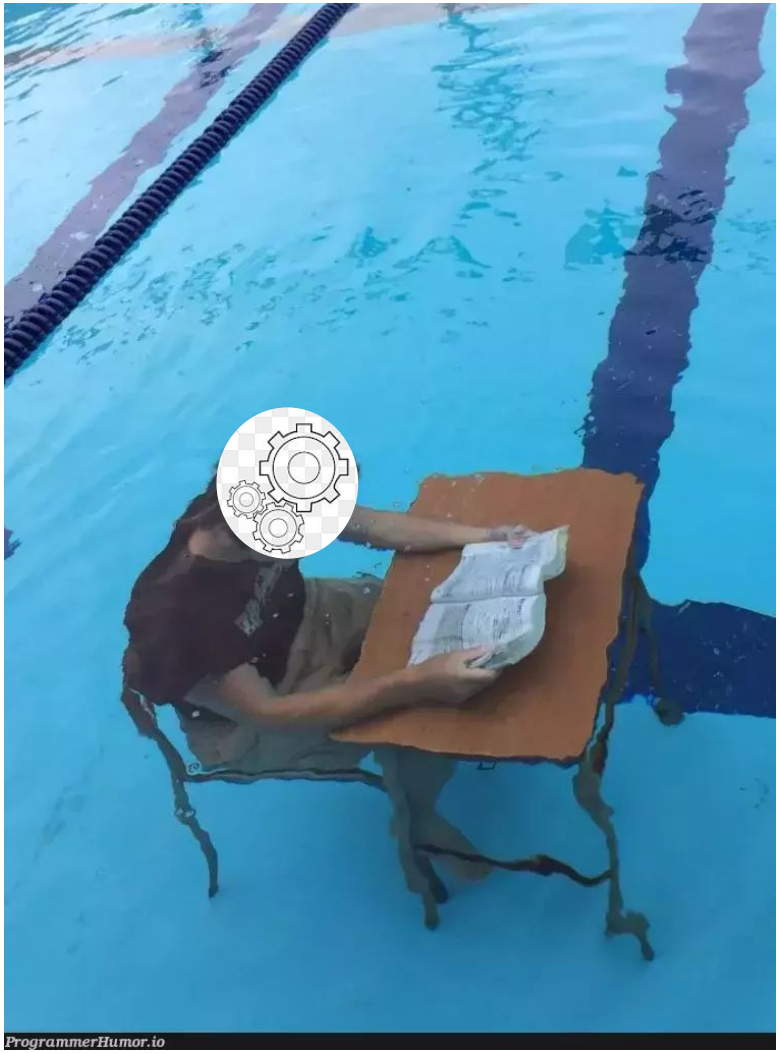
SyPy
Security & Privacy
Research Group

Deep Learning???

Basics of Machine Learning

SyPy
Security & Privacy
Research Group

Deep Learning???

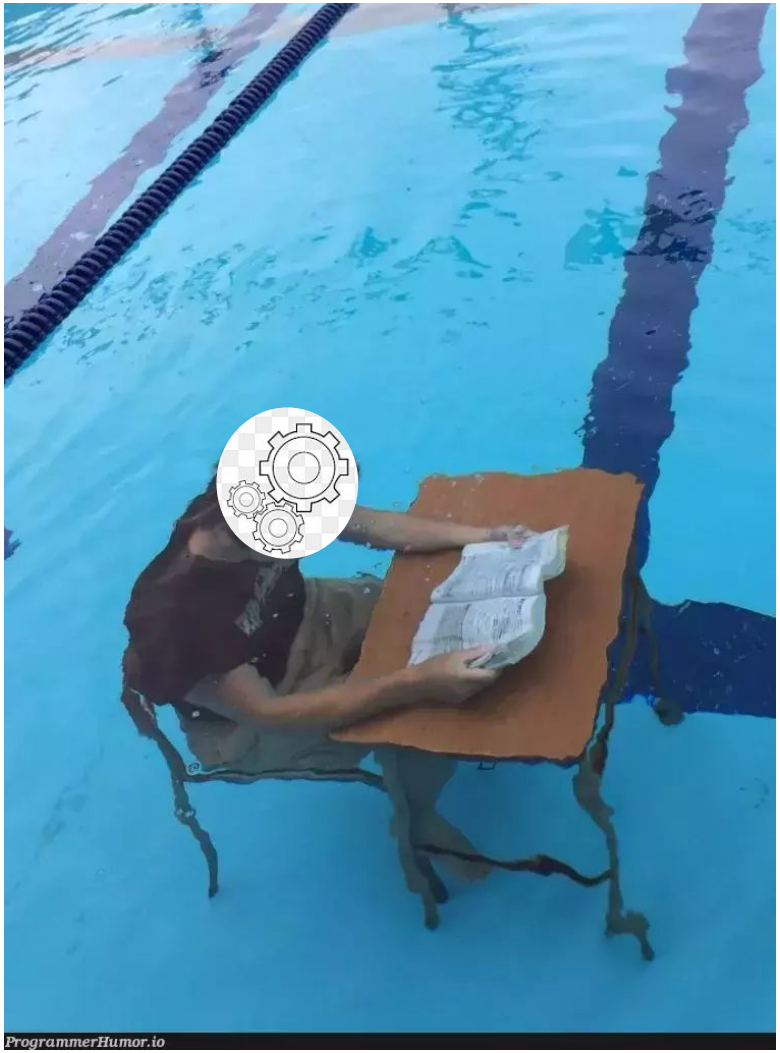


Basics of Machine Learning

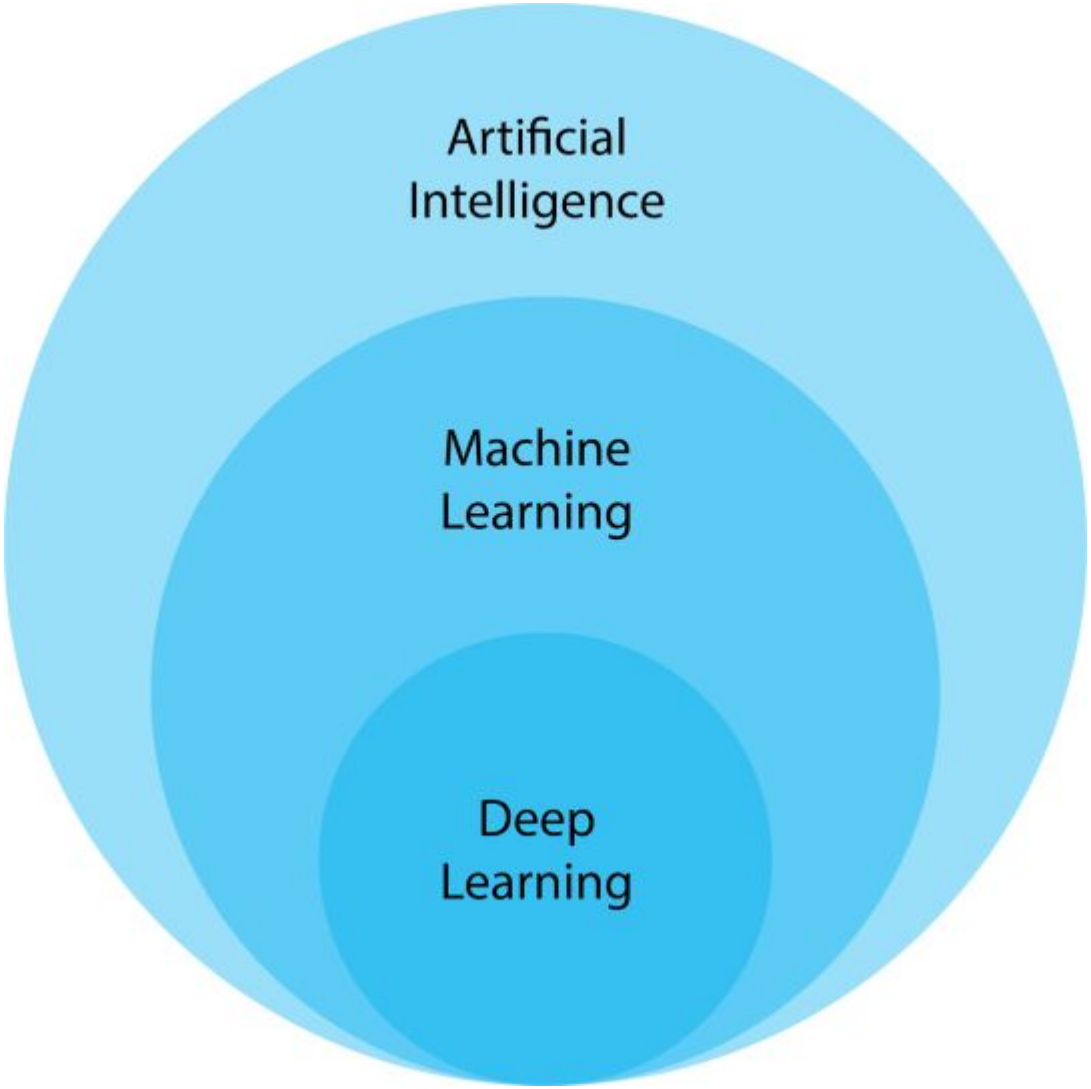
SyPy
Security & Privacy
Research Group

Deep Learning???

AGI??



Basics of Machine Learning



SyPy
Security & Privacy
Research Group

Why Machine Learning?

SyPy
Security & Privacy
Research Group

Why Machine Learning?

- Generalisable

SyPy
Security & Privacy
Research Group

Why Machine Learning?

- Generalisable
- Scalable

Basics of Machine Learning

SyPy
Security & Privacy
Research Group

Why Machine Learning?

- Generalisable
- Scalable
- Adaptable

Basics of Machine Learning

SyPy
Security & Privacy
Research Group

Why Machine Learning?

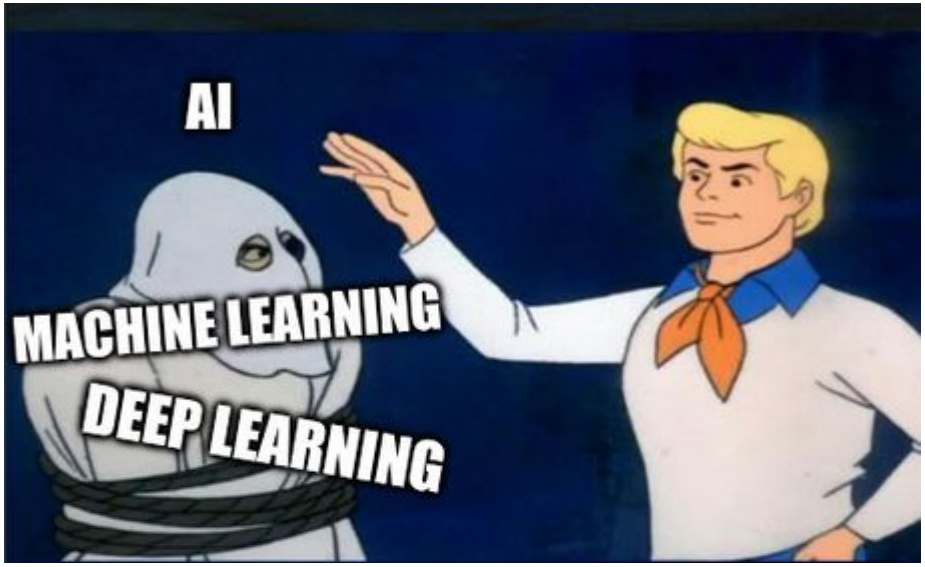
- Generalisable
- Scalable
- Adaptable

All you need is data! and compute

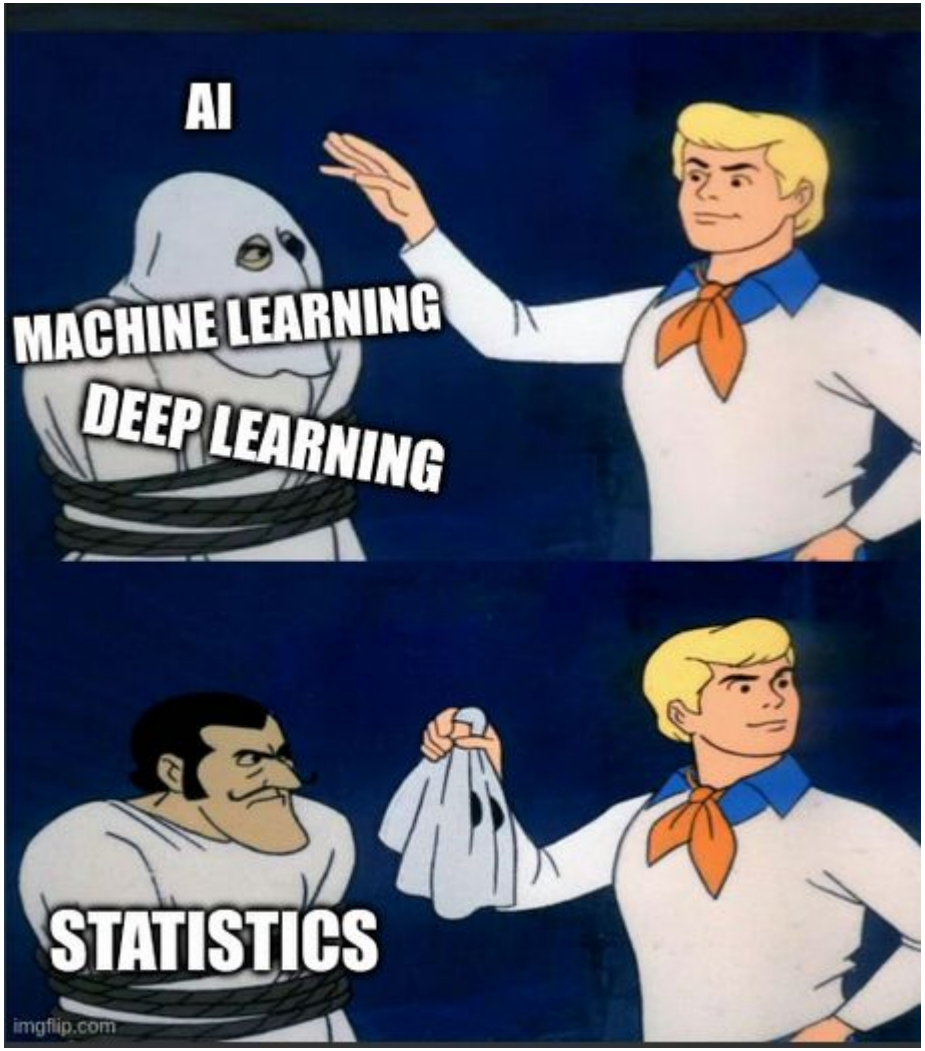
SyPy
Security & Privacy
Research Group

What is Machine Learning?

Basics of Machine Learning



Basics of Machine Learning



Basic Workflow in ML

SyPy
Security & Privacy
Research Group

What is the basic task of machine learning?

Basic Workflow in ML

SyPy
Security & Privacy
Research Group

What is the basic task of machine learning?

- Given a function $f(x)$ we want to *approximate* this function as closely as possible

Basic Workflow in ML

SyPy Security & Privacy Research Group

What is the basic task of machine learning?

- Given a function $f(x)$ we want to *approximate* this function as closely as possible
- An ML model is simply an approximation of this function

Basic Workflow in ML

SyPy Security & Privacy Research Group

What is the basic task of machine learning?

- Given a function $f(x)$ we want to *approximate* this function as closely as possible
- An ML model is simply an approximation of this function
- It can be designed to take any kind of input and any kind of output

Basic Workflow in ML

SyPy Security & Privacy Research Group

What is the basic task of machine learning?

- Given a function $f(x)$ we want to *approximate* this function as closely as possible
- An ML model is simply an approximation of this function
- It can be designed to take any kind of input and any kind of output
- Take the example of the YouTube recommendation algorithm…

Basic Workflow in ML

SyPy
Security & Privacy
Research Group

Q. How do we find the best estimate of $f(x)$?

Ans: We learn!

Basic Workflow in ML

SyPy
Security & Privacy
Research Group

Q. How do we find the best estimate of $f(x)$?

Ans: We learn!

Q. How do we learn?

Ans: By training (looking at previous examples, checking what could be improved, and repeating)

Basic Workflow in ML

SyPy Security & Privacy Research Group

Let's take the example of humans learning how to drive..

Week 1: Terrible

Week 2: Decent

Week 3: Good

Week 4: Expert

This is how ML models learn!!

Basic Workflow in ML

SyPy
Security & Privacy
Research Group

Now let's try to find the most important aspects of 'learning how to drive'

Task/Goal: Learn to drive	Task
Medium to learn: Driving Instructor	Algorithm
Experience: More you drive, better you get	Data

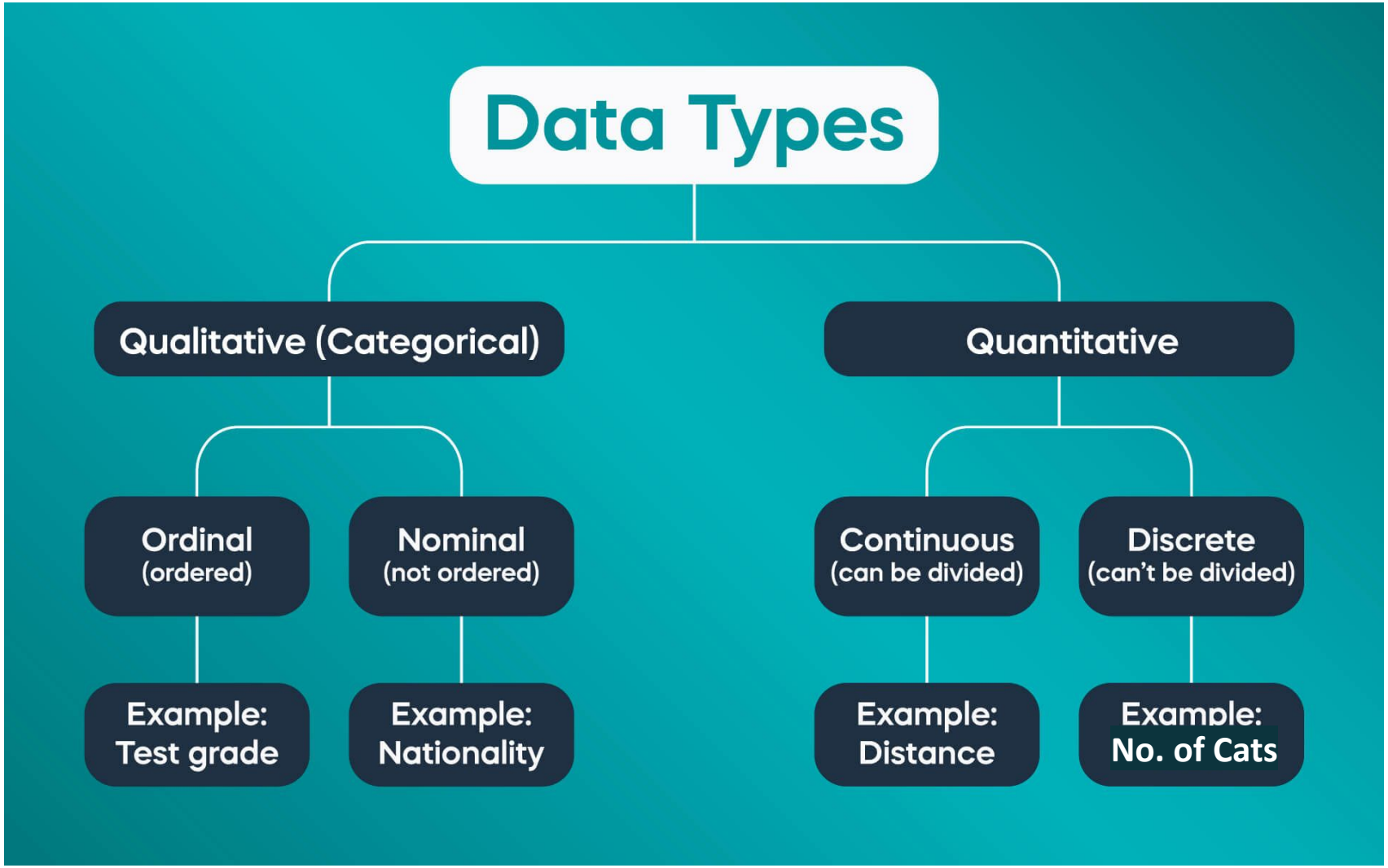
This is the basic approach to training an ML model

Some Terminology

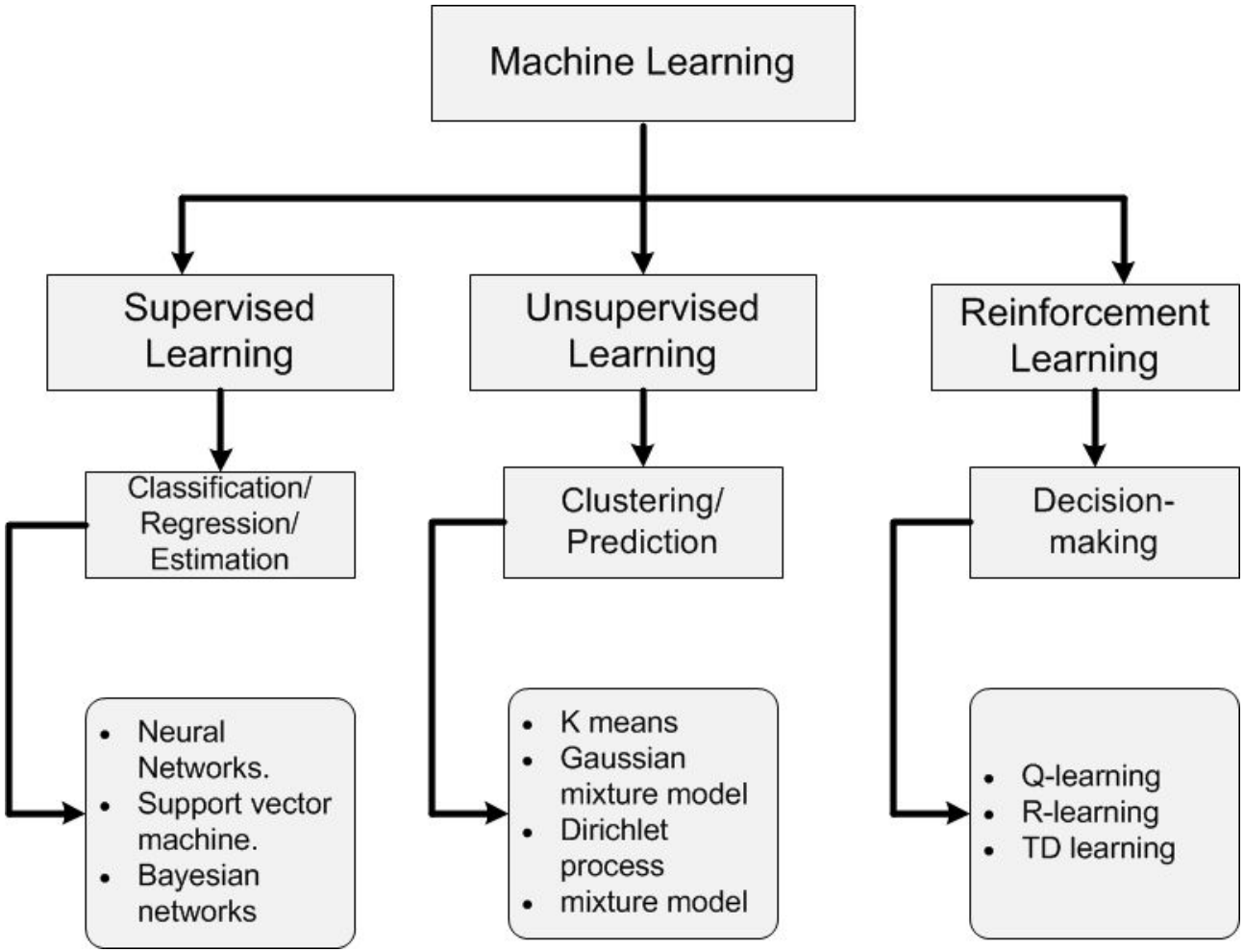
SyPy
Security & Privacy
Research Group

Now, let's look at some terminology before we explore more topics…

Taxonomy of Data



Taxonomy of ML Tasks



Regression

Predicts continuous numerical values

Examples:

- Stock market prediction
- Housing price prediction
- Estimated delivery times

Classification

SyPy
Security & Privacy
Research Group

Categorizes data into classes or groups

Examples:

- Image classification
- Video classification
- Hate speech detection
- Link prediction in social networks

Building a Neural Network

SyPy Security & Privacy Research Group

- Now, our goal is to build a function approximator such that it is learnable and can process input

- The neural network approach is to try to emulate the human brain using "layers" of processing

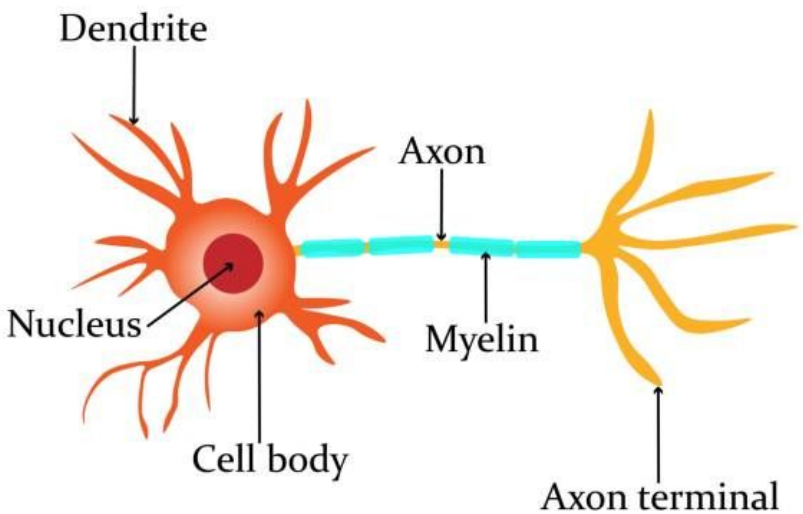
- To this end, we start with the smallest unit of the human brain: the humble neuron

Building a Neural network

SyPy
Security & Privacy
Research Group

BIOLOGY???

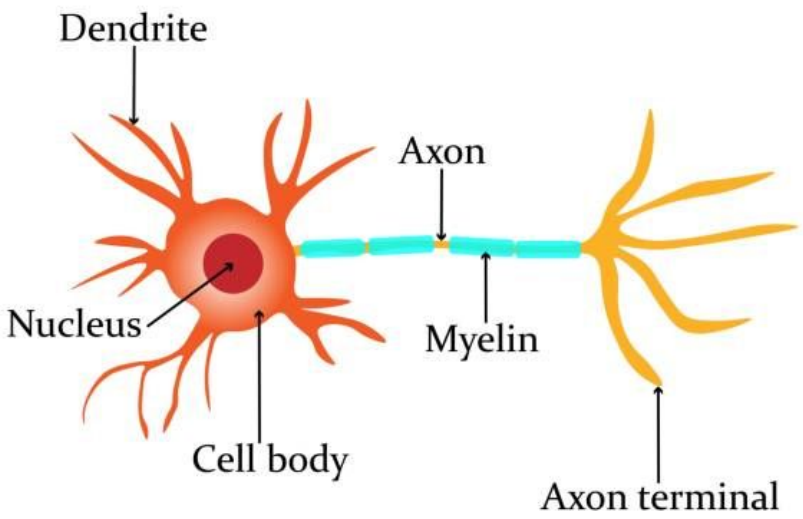
Neuron



3 main parts:

- Dendrites
- Body (soma, axon)
- Axon terminal

Neuron



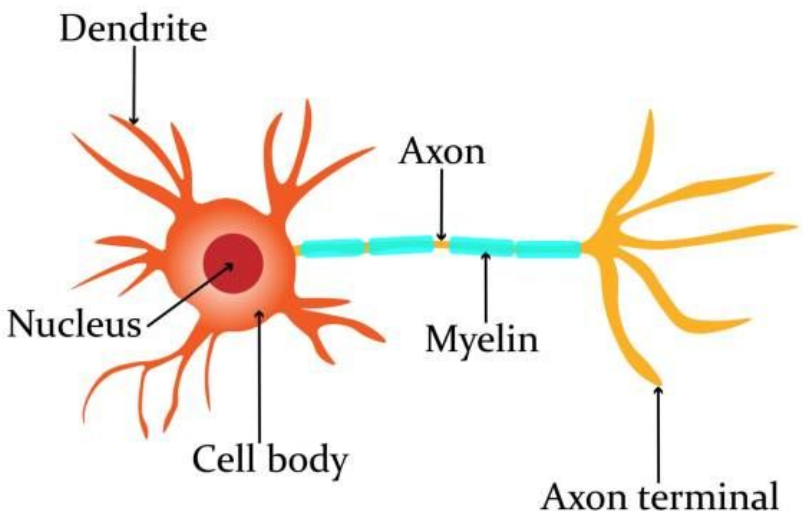
3 main parts:

- Dendrites
- Body (soma, axon)
- Axon terminal

This is similar to a function:

- Input
- Processing
- Output

Neuron



3 main parts:

- Dendrites
- Body (soma, axon)
- Axon terminal

This is similar to a function:

- Input
- Processing
- Output

Using this we can build our own version of a neuron, called a **perceptron**

Perceptron

SyPy Security & Privacy Research Group

Let us start with the a very simple mathematical function which can take an input, process it and produce an output

$$y = wx + b$$

But do you think such a simple equation can reproduce complicated functions?

Issues:

1. Lack of multiple inputs (features)
2. Linearity

We shall attempt to address these problems one by one

Multiple Input features

But what about multiple input features?

Multiple Input features

This is an easy fix. We can take in multiple input features, and simply assign weights to every single feature as follows:

Multiple Input features

SyPy
Security & Privacy
Research Group

This is an easy fix. We can take in multiple input features, and simply assign weights to every single feature as follows:

$$y = \sum_{i=1}^{n} w_i x_i + b$$

Multiple Input features

This is an easy fix. We can take in multiple input features, and simply assign weights to every single feature as follows:

$$y = \sum_{i=1}^{n} w_i x_i + b$$

We can simplify this representation using vectors and matrices

Multiple Input features

SyPy Security & Privacy Research Group

This is an easy fix. We can take in multiple input features, and simply assign weights to every single feature as follows:

$$y = \sum_{i=1}^{n} w_i x_i + b$$

We can simplify this representation using vectors and matrices

$$W = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_n \end{bmatrix} \quad X = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \qquad y = W^T X + b$$

Multiple Input features

SyPy Security & Privacy Research Group

This is an easy fix. We can take in multiple input features, and simply assign weights to every single feature as follows:

$$y = \sum_{i=1}^{n} w_i x_i + b$$

We can simplify this representation using vectors and matrices

$$W = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_n \end{bmatrix} \quad X = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \qquad y = W^T X + b$$

Wooo linear algebra

Multiple input features

Let's try to visualise…

$$y = \sum_{i=1}^{n} w_i x_i + b$$

Linearity

The capabilities of linear functions are limited

To mitigate this issue, we introduce **activation functions**

An activation function is a mathematical function applied to the output of a neuron

Examples include:

- Sigmoid
- tanh
- ReLU
- Softmax

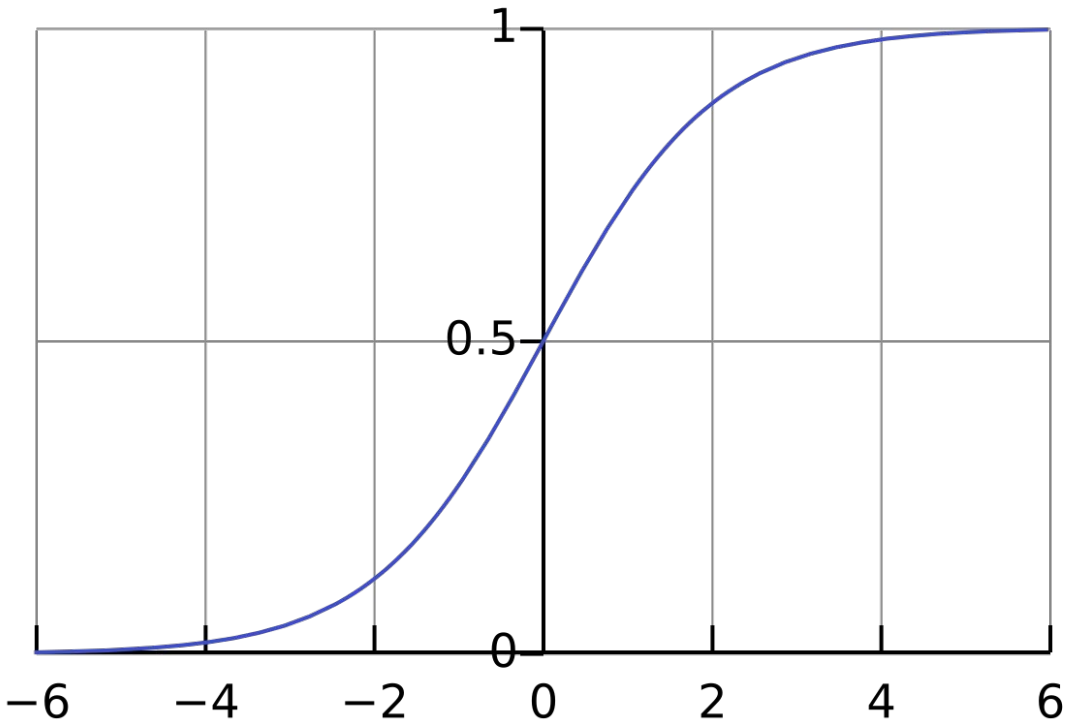
Activation Functions

Sigmoid Activation Function

$$\sigma(x) = \frac{1}{1+e^{-x}}$$

Characterized by it's S shape.

Output is restricted to between 0 and 1.



Activation Functions

Tanh Activation Function

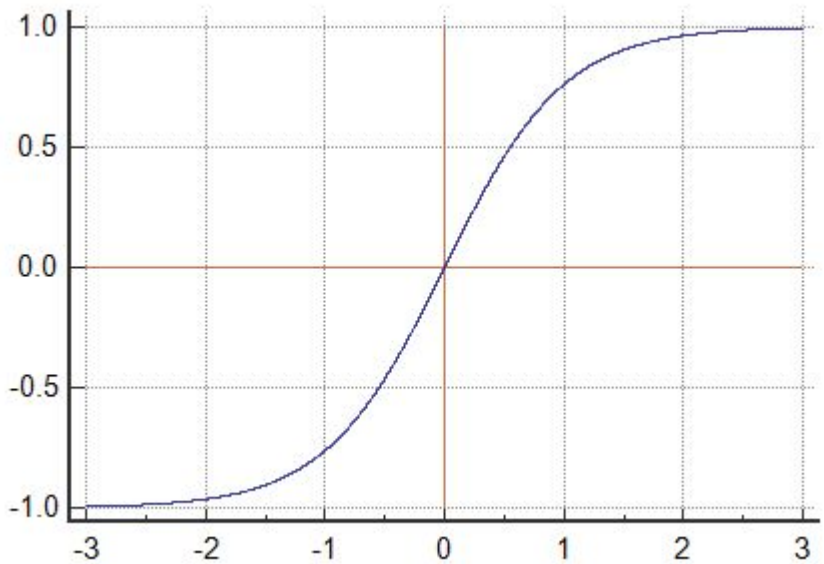
$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Shifted version of sigmoid that is stretched along the y-axis

$$\tanh(x) = 2 \times \text{sigmoid}(2x) - 1$$

Output is between -1 and 1.

0-centered distribution

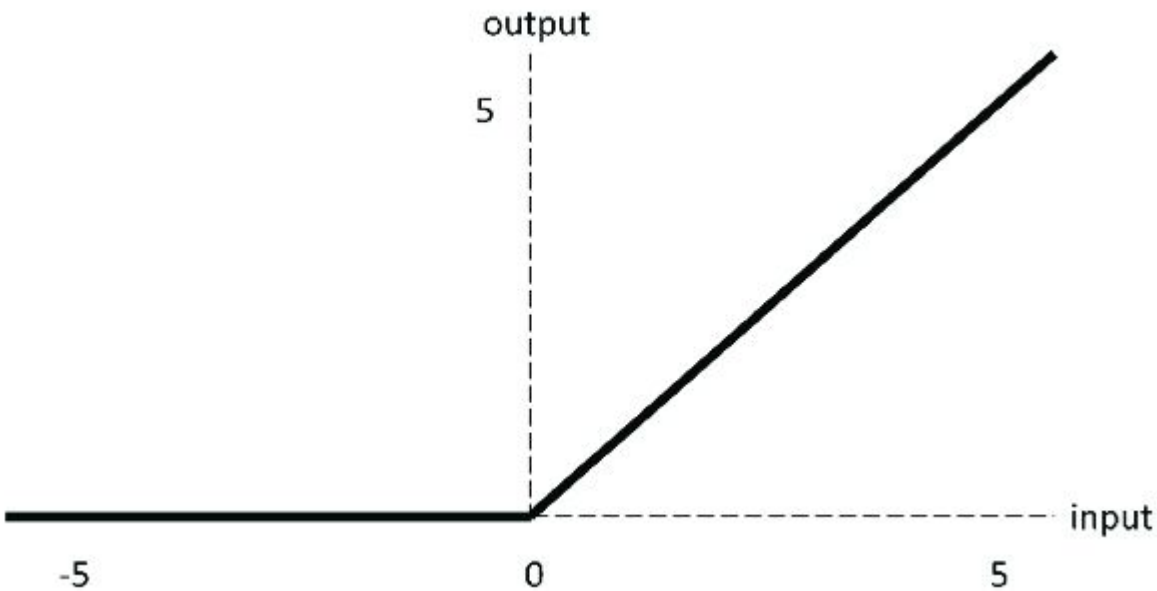


Activation Functions

ReLU Activation Function

$$\mathrm{ReLU}(x) = \max(0, x)$$

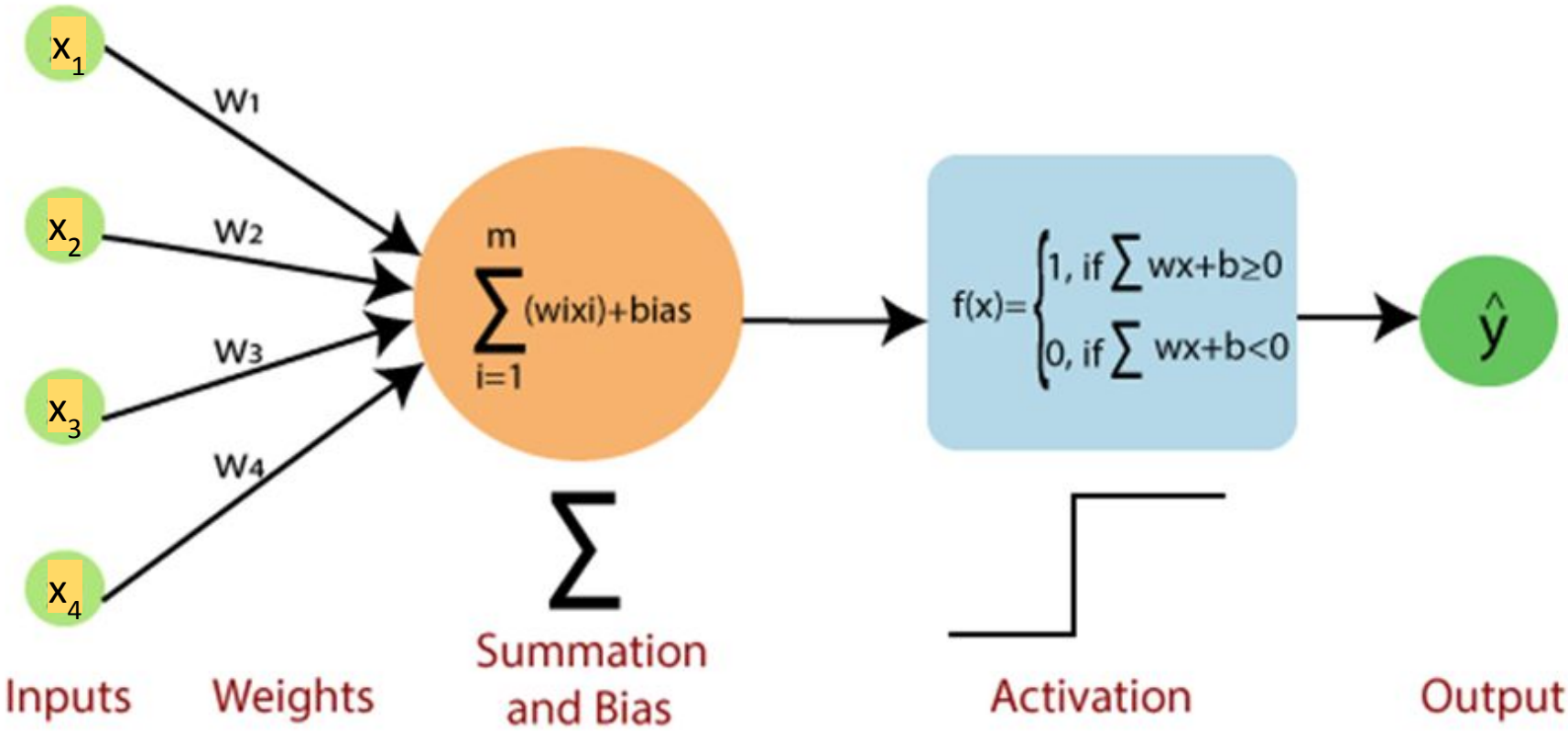
Output range is between 0 and -inf



Putting it together

Now we have the parts for our Perceptron 2.0, with new and improved features!

Let's put them all together in this example:

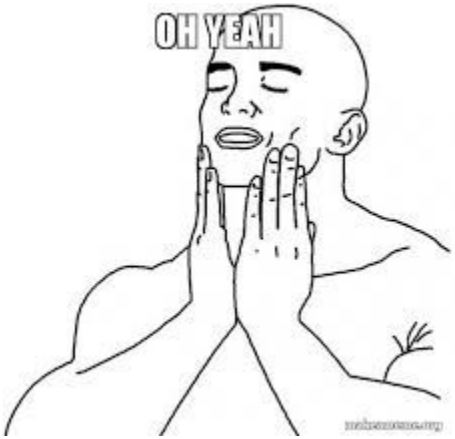


Continue putting it together

SyPy Security & Privacy Research Group

If we take multiple of these artificial *neurons* and put them together (making a *network*), we get one step closer to getting a:

Neural Network



To be completely precise, we've only constructed one 'layer' till now

Putting even more together?

If we keep putting together multiple similar layers, what we get is called a Multi-Layer Perceptron (MLP)

This is the most basic version of a Neural Network :D

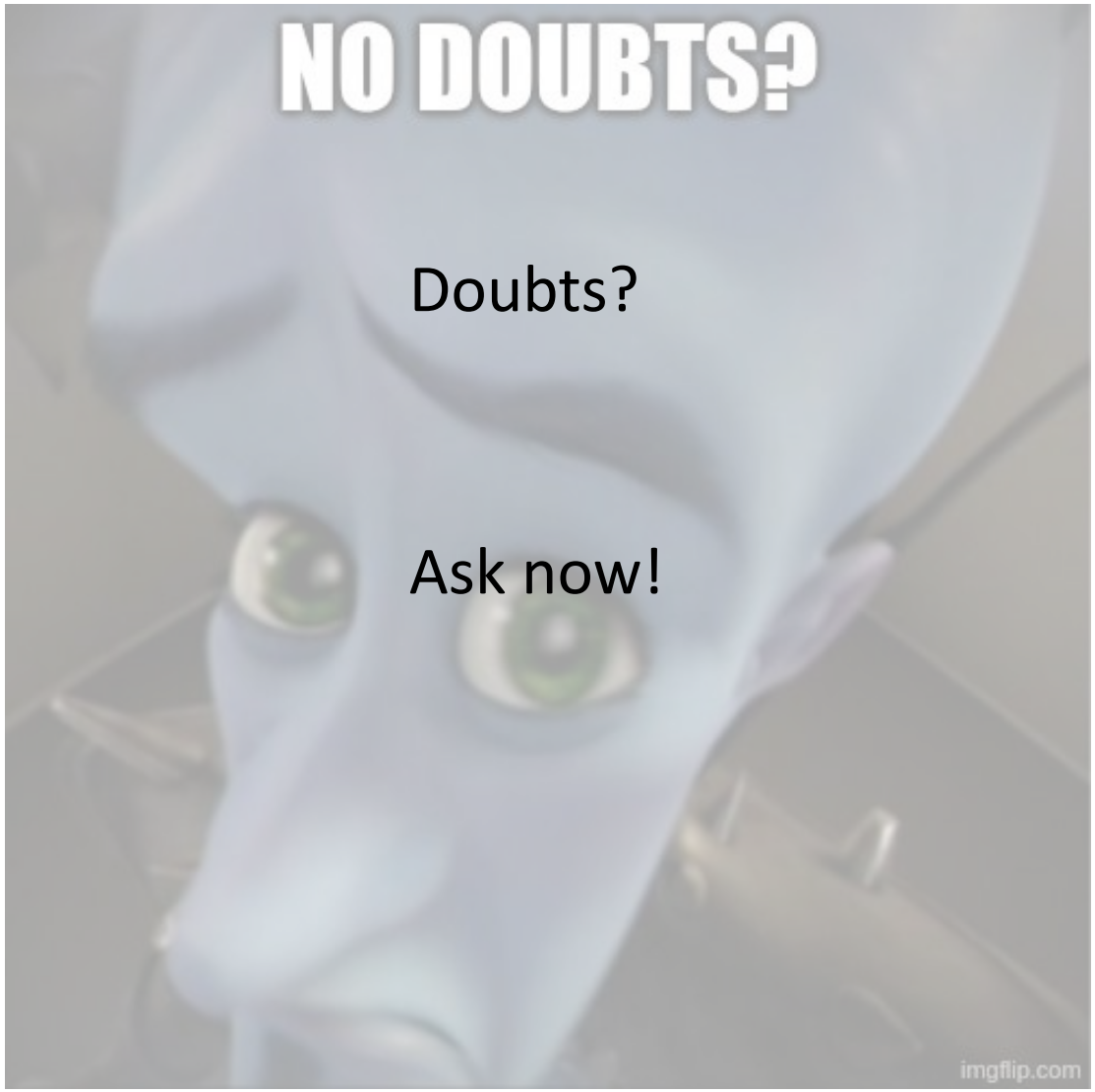
A moment to breathe…

SyPy
Security & Privacy
Research Group

Doubts?

Ask now!

Doubts?



Deep Neural Networks

Deep Neural Networks

Deep Neural Network (Multilayer Perceptron)

Why do we need deeper neural networks?

Deeper in this context refers to having additional layers

Universal approximation theorem

SyPy Security & Privacy Research Group

Universal approximation theorem:[13]—There exists an activation function σ which is analytic, strictly increasing and sigmoidal and has the following property: For any $f \in C[0,1]^d$ and $\varepsilon > 0$ there exist constants $d_i, c_{ij}, \theta_{ij}, \gamma_i$, and vectors $\mathbf{w}^{ij} \in \mathbb{R}^d$ for which

$$\left| f(\mathbf{x}) - \sum_{i=1}^{6d+3} d_i \sigma \left(\sum_{j=1}^{3d} c_{ij} \sigma(\mathbf{w}^{ij} \cdot \mathbf{x} - \theta_{ij}) - \gamma_i \right) \right| < \varepsilon$$

for all $\mathbf{x} = (x_1, \ldots, x_d) \in [0,1]^d$.

Universal approximation theorem

Universal approximation theorem:[13]—There exists an activation function σ which is analytic, strictly increasing and sigmoidal and has the following property: For any $f \in C[0,1]^d$ and $\varepsilon > 0$ there exist constants $d_i, c_{ij}, \theta_{ij}, \gamma_i$, and vectors $\mathbf{w}^{ij} \in \mathbb{R}^d$ for which

$$\left| f(\mathbf{x}) - \sum_{i=1}^{6d+3} d_i \sigma \left(\sum_{j=1}^{3d} c_{ij} \sigma(\mathbf{w}^{ij} \cdot \mathbf{x} - \theta_{ij}) - \gamma_i \right) \right| < \varepsilon$$

for all $\mathbf{x} = (x_1, \ldots, x_d) \in [0,1]^d$.

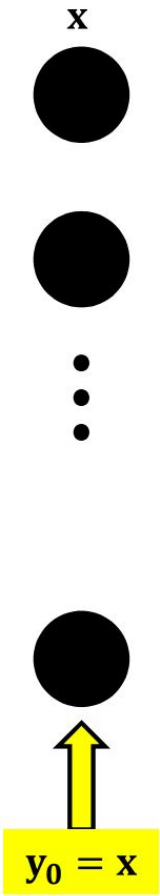
In simple words, *the universal approximation theorem says that there exists an activation function and a set of parameters (weights and biases) with which a neural network with at least two hidden layers can approximate any given function*

Deep Neural Network

Now, let's try to build a simple feedforward neural network with multiple hidden layers

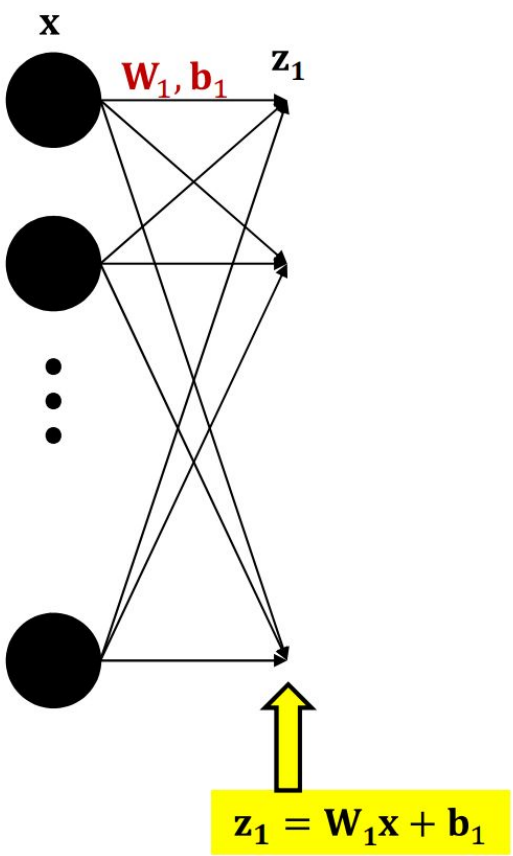
SyPy Security & Privacy Research Group

The forward pass: Evaluating the network



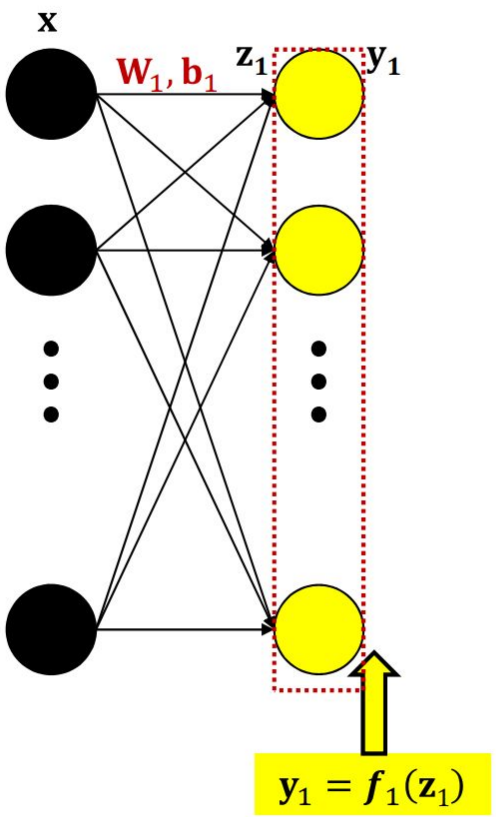
SyPy Security & Privacy Research Group

The forward pass



SyPy Security & Privacy Research Group

The forward pass



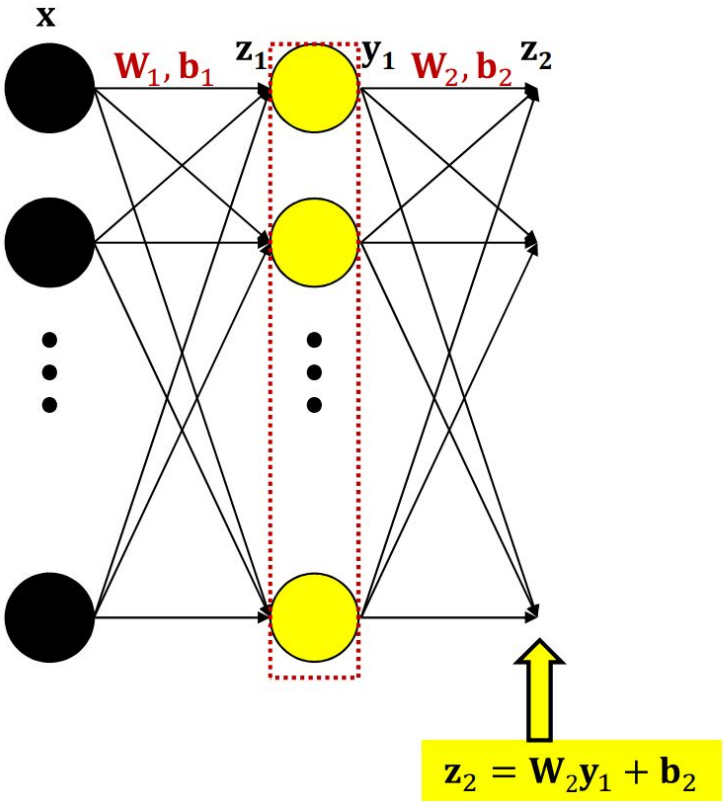
The Complete computation

$$\mathbf{y}_1 = f_1(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1)$$

38

SyPy Security & Privacy Research Group

The forward pass



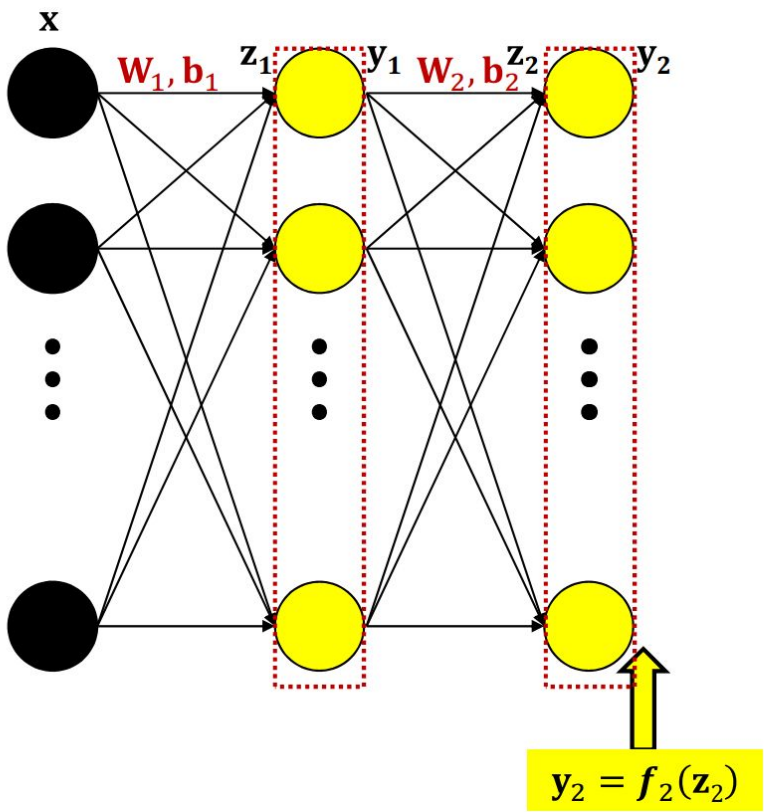
The Complete computation

$$y_1 = f_1(W_1 x + b_1)$$

39

SyPy Security & Privacy Research Group

The forward pass



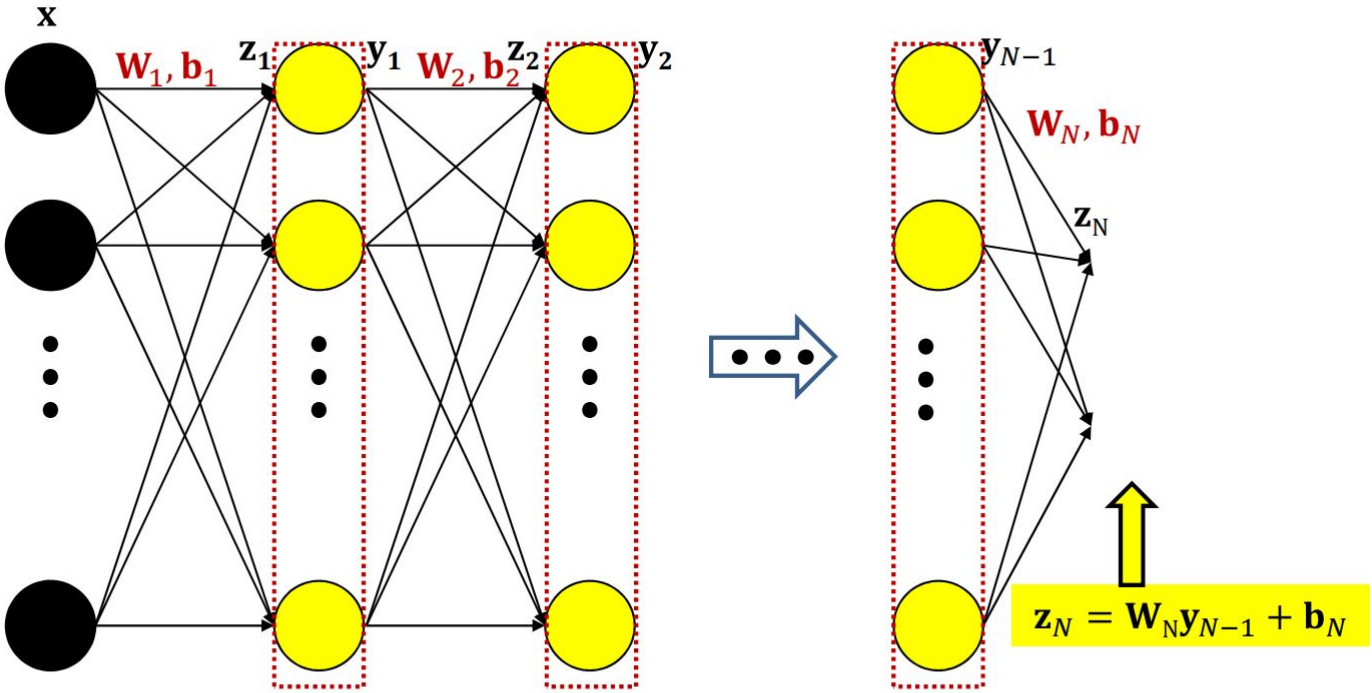
The Complete computation

$$y_2 = f_2(W_2 f_1(W_1 x + b_1) + b_2)$$

40

SyPy Security & Privacy Research Group

The forward pass



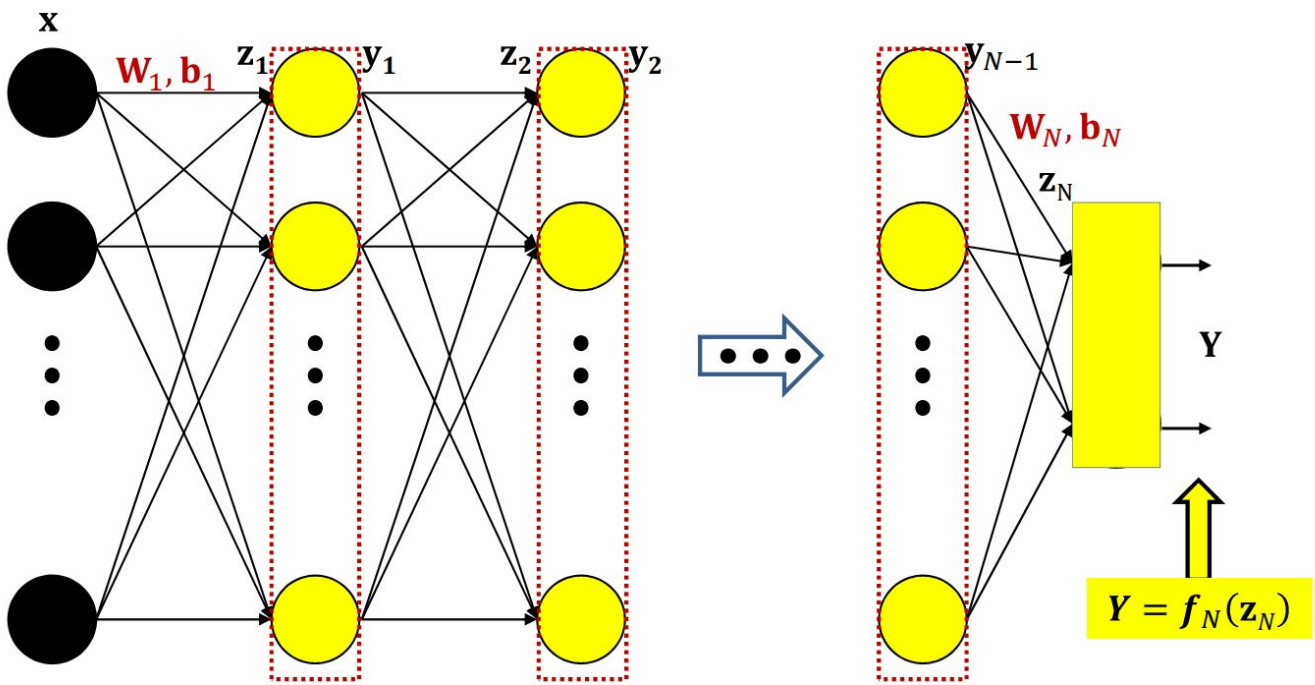
The Complete computation

$$y_2 = f_2(W_2 f_1(W_1 x + b_1) + b_2)$$

41

SyPy Security & Privacy Research Group

The forward pass



The Complete computation

$$Y = f_N(\mathbf{W}_N f_{N-1}(\dots f_2(\mathbf{W}_2 f_1(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2)\dots) + \mathbf{b}_N)$$

42

SyPy Security & Privacy Research Group

The Forward Pass

- Set $\mathbf{y}_0 = \mathbf{x}$
- Recursion through layers:
  - For layer k = 1 to N:

$$\mathbf{z}_k = \mathbf{W}_k \mathbf{y}_{k-1} + \mathbf{b}_k$$
$$\mathbf{y}_k = \boldsymbol{f}_k(\mathbf{z}_k)$$

- Output:

$$\mathbf{Y} = \mathbf{y}_N$$

Designing Neural Networks

SyPy
Security & Privacy
Research Group

How do you design an MLP for a:

- Regression task?
- Classification task?

What will be the outputs?

Training a Neural Network

SyPy
Security & Privacy
Research Group

Now, we have a model that can potentially act as an approximator for any desired function, given the right weights

But where do these weights come from?

Revisiting how humans learn a task…

Whenever we make a mistake, we try to learn from it and adjust ourselves accordingly (ideally). But this needs a ‘metric’ that tells us how far off we are from the right thing!

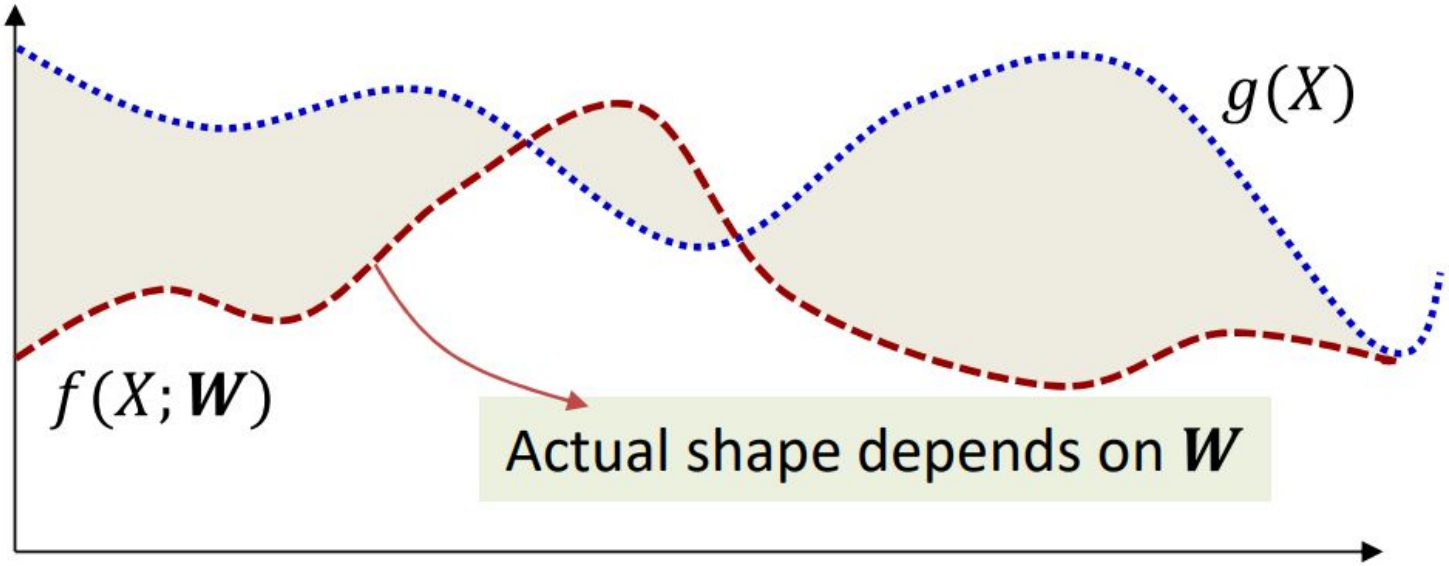
Training a Neural Network

For MLPs also we need to define some metric that gives us the measure of how far off we are from the correct output

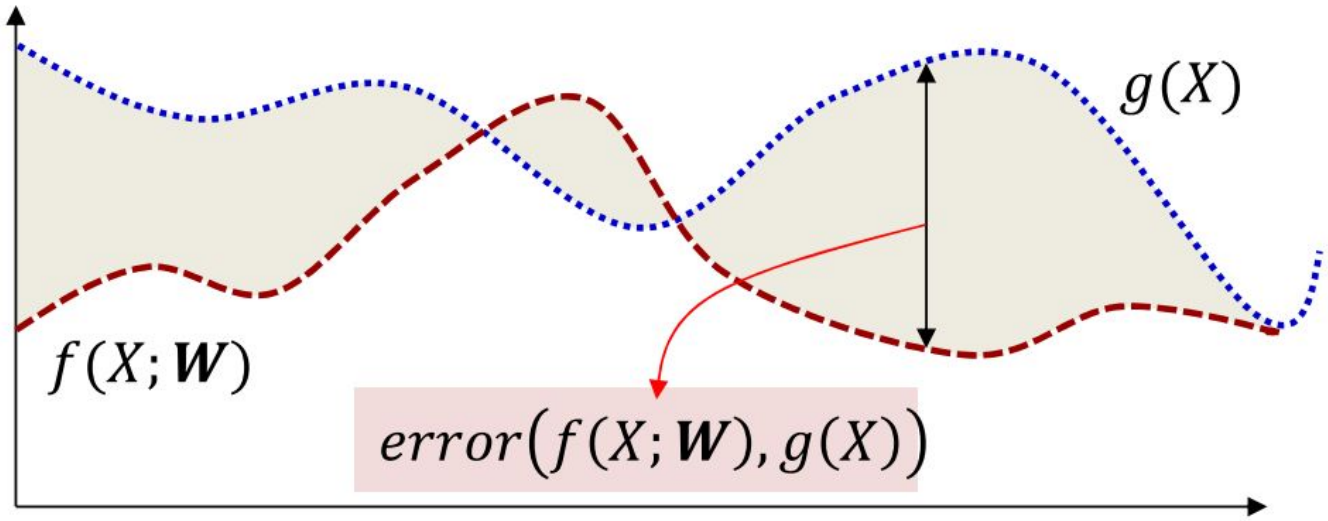
We call this the **loss** (or the **cost** or the **objective function**)

Loss Functions

For a given set of parameters ***W***, our approximator (model) would have a certain behaviour



Loss Functions



$$totalerr(\boldsymbol{W}) = \int_{-\infty}^{\infty} error\big(f(X;\boldsymbol{W}), g(X)\big) dX$$

$$\widehat{\boldsymbol{W}} = \underset{\boldsymbol{W}}{\operatorname{argmin}}\ totalerr(\boldsymbol{W})$$

Loss Functions

Regression

SyPy Security & Privacy Research Group

- Mean Square Error

$$\mathcal{L}_{mse} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

- Mean Absolute Error

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

- Relative Mean Square Error

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

Loss Functions

Classification

SyPy Security & Privacy Research Group

- Binary Cross Entropy loss

$$\mathrm{BCE} = -\frac{1}{n}\sum_{i=1}^{n}\left[y_i \log(\hat{y}_i) + (1-y_i)\log(1-\hat{y}_i)\right]$$

- Cross Entropy loss

$$\mathrm{CE} = -\frac{1}{n}\sum_{i=1}^{n}\sum_{k=1}^{K} y_{i,k}\log(\hat{y}_{i,k})$$

Loss Functions

Classification

SyPy
Security & Privacy
Research Group

Wait a minute?!

Loss Functions

Classification

SyPy
Security & Privacy
Research Group

Wait a minute?!

Entropy?

Loss Functions

Classification

SyPy
Security & Privacy
Research Group

Wait a minute?!

Entropy?

The one from thermodynamics??

Loss Functions

Classification

SyPy
Security & Privacy
Research Group

Wait a minute?!

Entropy?

The one from thermodynamics??

Well, yes and no…

SyPy Security & Privacy Research Group

Detour: A Very Short Introduction to Information Theory

- What is Information?
  - Let us look at some examples…

- Which of these is more informative?

(a) Snowfall in Mount Everest

(b) Snowfall in Hyderabad

https://telanganatoday.com/rarest-of-rare-when-villages-in-telangana-got-a-snowy-dressing

SyPy Security & Privacy Research Group

- Which of these is more informative?

(a) Contaminated food in Kadamba (b) Clean food in Kadamba

SyPy Security & Privacy Research Group

- Which of these is more informative?

(a) Getting good marks in IIS quiz (b) Getting bad marks in IIS quiz

- Our intuition says that the information (gain) is inversely proportional to the probability of the event, i.e., the more likely the event, the less information we gain from it, and vice-versa.

- Our intuition says that the information (gain) is inversely proportional to the probability of the event, i.e., the more likely the event, the less information we gain from it, and vice-versa.
- For an event x ,

$$h(x) \propto \frac{1}{p(x)}$$

where $h(x)$ is the information gained from event x and $p(x)$ is probability of event x .

Cross Entropy

- Shannon Information

$$h(x) = \log_2(\frac{1}{p(x)}) = -\log_2(p(x))$$

SyPy Security & Privacy Research Group

Cross Entropy

- Shannon Information

$$h(x) = \log_2(\frac{1}{p(x)}) = -\log_2(p(x))$$

- "the number of bits required to encode and transmit an event"
- More precisely, it is the theoretical minimum number of bits required (on average) for encoding and transmitting the event.

SyPy Security & Privacy Research Group

Cross Entropy

- Shannon Information

$$h(x) = \log_2(\frac{1}{p(x)}) = -\log_2(p(x))$$

- "the number of bits required to encode and transmit an event"
- More precisely, it is the theoretical minimum number of bits required (on average) for encoding and transmitting the event.

- For example,
  - encoding outcome of a fair coin would require a minimum of $-\log(\frac{1}{2}) = 1$ bits for (efficient) transmission

Cross Entropy

- Shannon Information

$$h(x) = \log_2(\frac{1}{p(x)}) = -\log_2(p(x))$$

- "the number of bits required to encode and transmit an event"
- More precisely, it is the theoretical minimum number of bits required (on average) for encoding and transmitting the event.

- For example,
  - encoding outcome of a fair coin would require a minimum of $-\log(\frac{1}{2}) = 1$ bits for (efficient) transmission
  - encoding outcome of a biased coin (p(H) = 1/4) would require a minimum of bits $-\log(\frac{1}{4}) = 2$ for (efficient) transmission

Cross Entropy

- Shannon Information

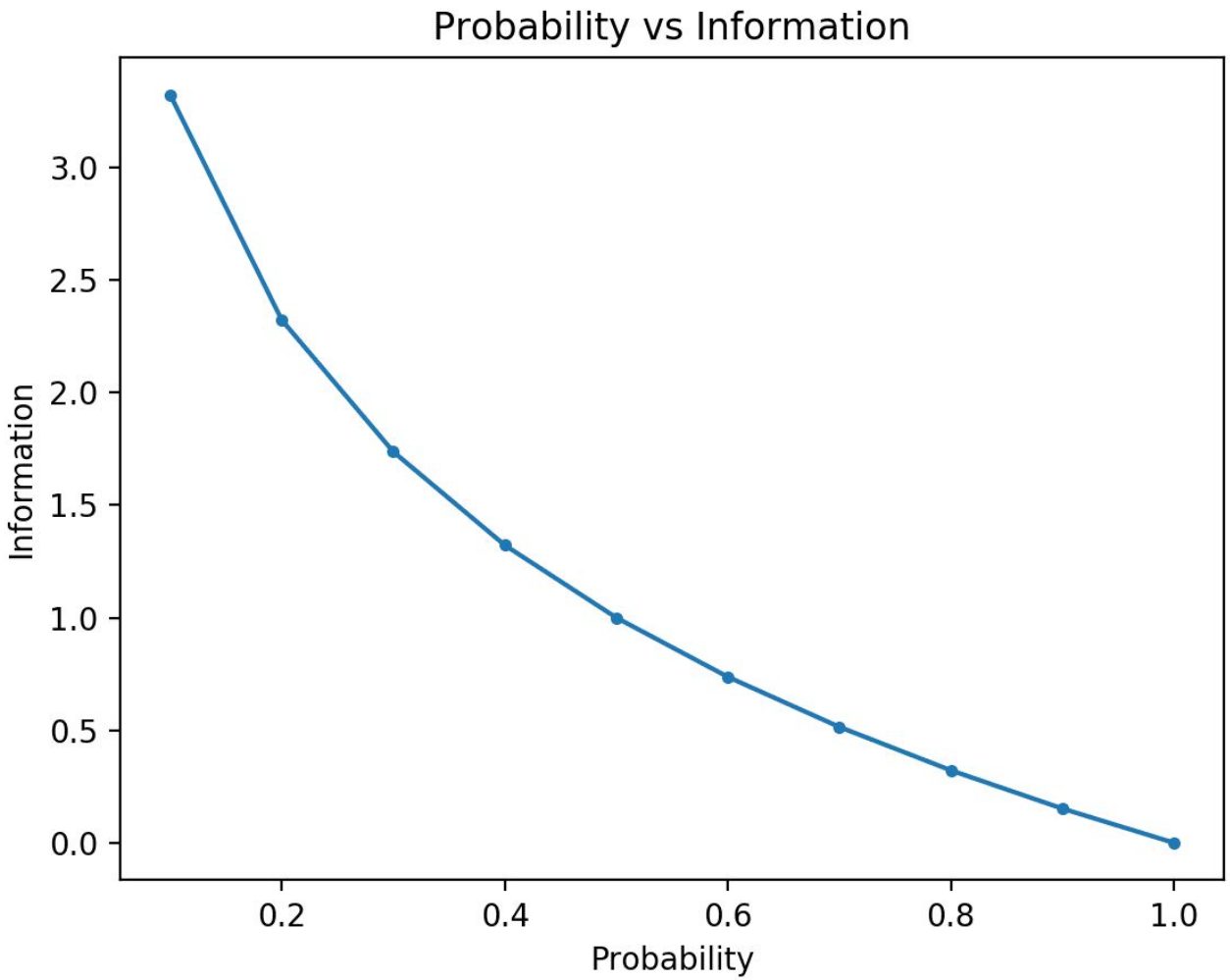
$$h(x) = \log_2(\frac{1}{p(x)}) = -\log_2(p(x))$$

- "the number of bits required to encode and transmit an event"
- More precisely, it is the theoretical minimum number of bits required (on average) for encoding and transmitting the event.

- For example,
  - encoding outcome of a fair coin would require a minimum of $-\log(\frac{1}{2}) = 1$ bits for (efficient) transmission
  - encoding outcome of a biased coin (p(H) = 1/4) would require a minimum of bits $-\log(\frac{1}{4}) = 2$ for (efficient) transmission
- [Optional] Reason for why log in the equation: https://perso.telecom-paristech.fr/rioul/publis/201811rioul.pdf

Basics of Machine Learning

Cross Entropy



SyPy Security & Privacy Research Group

- What is Entropy?
  - "… the Shannon entropy of a distribution is the expected amount of information in an event drawn from that distribution. It gives a lower bound on the number of bits […] needed on average to encode symbols drawn from a distribution P." Page 74, Deep Learning, 2016.

- (Shannon) Entropy
  - "the Shannon entropy of a distribution is the expected amount of information in an event drawn from that distribution. It gives a lower bound on the number of bits […] needed on average to encode symbols drawn from a distribution P."

- (Shannon) Entropy
  - "the Shannon entropy of a distribution is the expected amount of information in an event drawn from that distribution. It gives a lower bound on the number of bits [...] needed on average to encode symbols drawn from a distribution P."

$$H(X) = \mathbb{E}[h(X)] = -\sum_{x \in \mathcal{X}} p(x) \log_2 p(x),$$

SyPy Security & Privacy Research Group

- (Shannon) Entropy
  - "the Shannon entropy of a distribution is the expected amount of information in an event drawn from that distribution. It gives a lower bound on the number of bits [...] needed on average to encode symbols drawn from a distribution P."

$$H(X) = \mathbb{E}[h(X)] = -\sum_{x \in \mathcal{X}} p(x) \log_2 p(x),$$

  - e.g., to (efficiently) encode heads and tails drawn from a fair coin, one requires $-\frac{1}{2}\log_2(\frac{1}{2}) - \frac{1}{2}\log_2(\frac{1}{2}) = 1$ bits on average

- (Shannon) Entropy
  - "the Shannon entropy of a distribution is the expected amount of information in an event drawn from that distribution. It gives a lower bound on the number of bits [...] needed on average to encode symbols drawn from a distribution P."

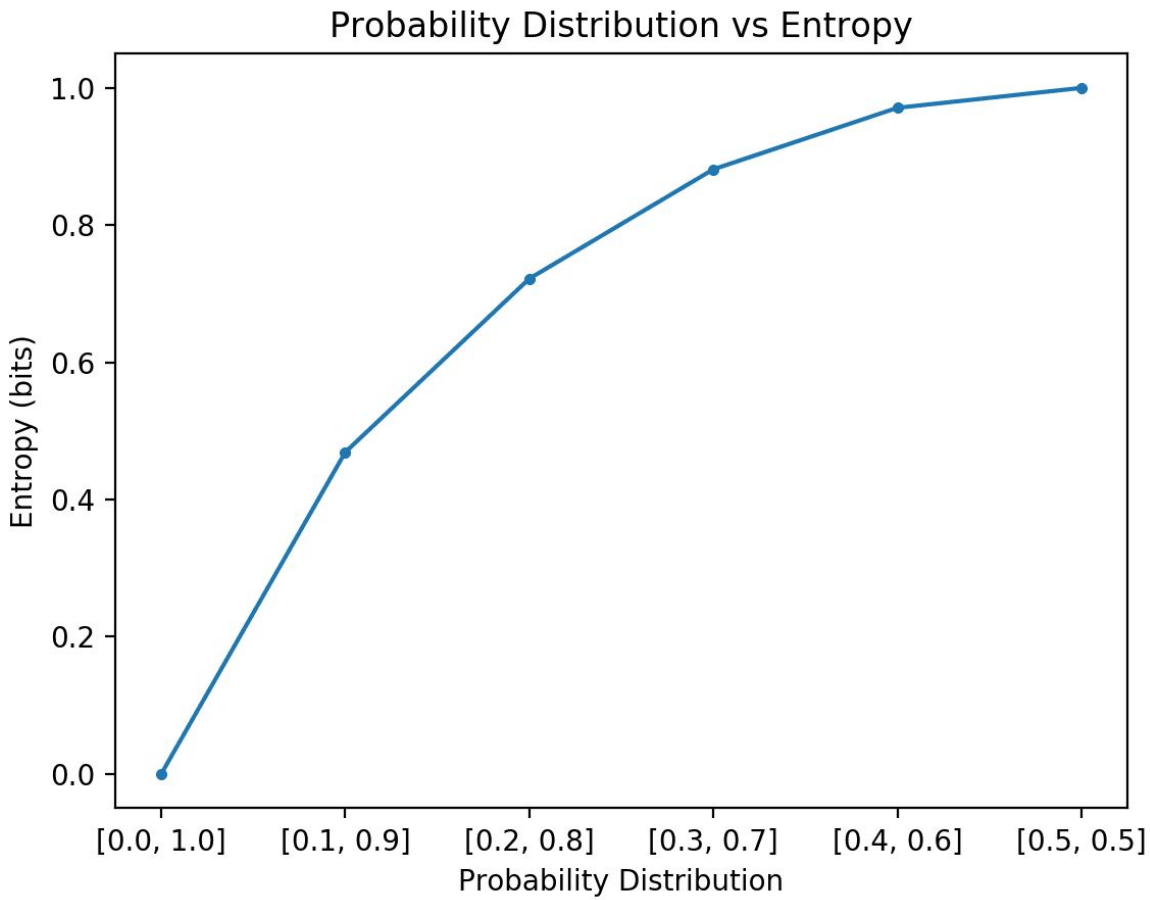
$$H(X) = \mathbb{E}[h(X)] = -\sum_{x \in \mathcal{X}} p(x) \log_2 p(x),$$

  - e.g., to (efficiently) encode heads and tails drawn from a fair coin, one requires $-\frac{1}{2}\log_2(\frac{1}{2}) - \frac{1}{2}\log_2(\frac{1}{2}) = 1$ bits on average
  - e.g., to (efficiently) encode heads (¼) and tails (¾) drawn from a biased coin, one requires $-\frac{1}{4}\log_2(\frac{1}{4}) - \frac{3}{4}\log_2(\frac{3}{4}) = 0.811$ bits on average

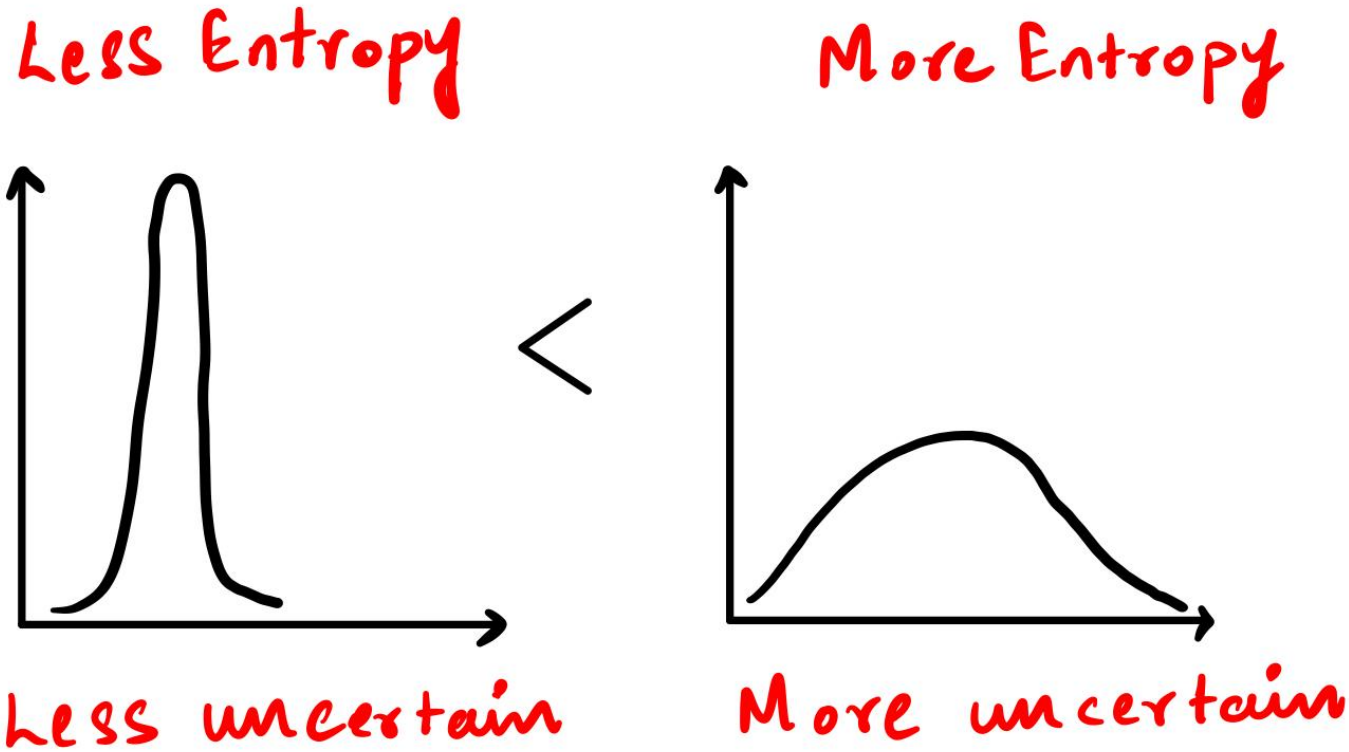
Basics of Machine Learning

Cross Entropy

For a coin toss, the entropy is maximum at equal probabilities (fair coin)



Cross Entropy



Cross Entropy

- What if you use a distribution different from the one which the message has been generated, to encode the message?

Cross Entropy

- What if you use a distribution different from the one which the message has been generated, to encode the message?
  - Special coin = (Head, Edge, Tail) = (H, E, T) = (1/4, 1/2, 1/4)

Cross Entropy

- What if you use a distribution different from the one which the message has been generated, to encode the message?
  - Special coin = (Head, Edge, Tail) = (H, E, T) = (1/4, 1/2, 1/4)
  - Message = EETHEHET

Cross Entropy

- What if you use a distribution different from the one which the message has been generated, to encode the message?
  - Special coin = (Head, Edge, Tail) = (H, E, T) = (1/4, 1/2, 1/4)
  - Message = EETHEHET
  - Fair coin = (H, E, T) = (1/3, 1/3, 1/3)

- What if you use a distribution different from the one which the message has been generated, to encode the message?
  - Special coin = (Head, Edge, Tail) = (H, E, T) = (1/4, 1/2, 1/4)
  - Message = EETHEHET
  - Fair coin = (H, E, T) = (1/3, 1/3, 1/3)
  - Bits required to encode fair coin = ceil($-\log_2(1/3)$) = ceil(1.585) = 2

Cross Entropy

- What if you use a distribution different from the one which the message has been generated, to encode the message?
  - Special coin = (Head, Edge, Tail) = (H, E, T) = (1/4, 1/2, 1/4)
  - Message = EETHEHET
  - Fair coin = (H, E, T) = (1/3, 1/3, 1/3)
  - Bits required to encode fair coin = ceil($-\log_2(1/3)$) = ceil(1.585) = 2
  - Let (H, E, T) = (00, 01, 10)

Cross Entropy

- What if you use a distribution different from the one which the message has been generated, to encode the message?
  - Special coin = (Head, Edge, Tail) = (H, E, T) = (1/4, 1/2, 1/4)
  - Message = EETHEHET
  - Fair coin = (H, E, T) = (1/3, 1/3, 1/3)
  - Bits required to encode fair coin = ceil($-\log_2(1/3)$) = ceil(1.585) = 2
  - Let (H, E, T) = (00, 01, 10)
  - Average number of bits per symbol = (8*2)/8 = 2

Cross Entropy

- What if you use a distribution different from the one which the message has been generated, to encode the message?
  - Special coin = (Head, Edge, Tail) = (H, E, T) = (1/4, 1/2, 1/4)
  - Message = EETHEHET
  - Fair coin = (H, E, T) = (1/3, 1/3, 1/3)
  - Bits required to encode fair coin = ceil($-\log_2(1/3)$) = ceil(1.585) = 2
  - Let (H, E, T) = (00, 01, 10)
  - Average number of bits per symbol = (8*2)/8 = 2
  - Optimal encoding = (H, E, T) = (2, 1, 2) bits = (10, 0, 11) or similar prefix

Cross Entropy

- What if you use a distribution different from the one which the message has been generated, to encode the message?
  - Special coin = (Head, Edge, Tail) = (H, E, T) = (1/4, 1/2, 1/4)
  - Message = EETHEHET
  - Fair coin = (H, E, T) = (1/3, 1/3, 1/3)
  - Bits required to encode fair coin = ceil($-\log_2(1/3)$) = ceil(1.585) = 2
  - Let (H, E, T) = (00, 01, 10)
  - Average number of bits per symbol = (8*2)/8 = 2
  - Optimal encoding = (H, E, T) = (2, 1, 2) bits = (10, 0, 11) or similar prefix
  - Average number of bits per symbol with optimal encoding = (4*2+4*1)/8 = 1.5 < 1.585 < 2

- What if you use a distribution different from the one which the message has been generated, to encode the message?
  - Special coin = (Head, Edge, Tail) = (H, E, T) = (1/4, 1/2, 1/4)
  - Message = EETHEHET
  - Fair coin = (H, E, T) = (1/3, 1/3, 1/3)
  - Bits required to encode fair coin = ceil($-\log_2(1/3)$) = ceil(1.585) = 2
  - Let (H, E, T) = (00, 01, 10)
  - Average number of bits per symbol = (8*2)/8 = 2
  - Optimal encoding = (H, E, T) = (2, 1, 2) bits = (10, 0, 11) or similar prefix
  - Average number of bits per symbol with optimal encoding = (4*2+4*1)/8 = 1.5 < 1.585 < 2
- Average number of bits required to encode symbol is greater

- THIS is cross entropy
  - "… the cross entropy is the average number of bits needed to encode data coming from a source with distribution p when we use model q …" Page 57, Machine Learning: A Probabilistic Perspective, 2012.

SyPy Security & Privacy Research Group

- THIS is cross entropy
  - "… the cross entropy is the average number of bits needed to encode data coming from a source with distribution p when we use model q …" Page 57, Machine Learning: A Probabilistic Perspective, 2012.
  - $$H(p, q) = -\sum_{x \in \mathcal{X}} p(x) \log_2 q(x)$$

- KL divergence
  - "In other words, the KL divergence is the average number of extra bits needed to encode the data, due to the fact that we used distribution *q* to encode the data instead of the true distribution *p*." Page 58, Machine Learning: A Probabilistic Perspective, 2012.

SyPy Security & Privacy Research Group

- KL divergence
  - "In other words, the KL divergence is the average number of extra bits needed to encode the data, due to the fact that we used distribution *q* to encode the data instead of the true distribution *p*." Page 58, Machine Learning: A Probabilistic Perspective, 2012.
  - $$D_{\mathrm{KL}}(p \parallel q) = H(p, q) - H(p) = -\sum_{x \in \mathcal{X}} p(x) \ \log_2 \left(\frac{q(x)}{p(x)} \right)$$

SyPy Security & Privacy Research Group

- KL divergence
  - "In other words, the KL divergence is the average number of extra bits needed to encode the data, due to the fact that we used distribution *q* to encode the data instead of the true distribution *p*." Page 58, Machine Learning: A Probabilistic Perspective, 2012.
  - $$D_{\mathrm{KL}}(p \parallel q) = H(p,q) - H(p) = -\sum_{x \in \mathcal{X}} p(x) \ \log_2 \left(\frac{q(x)}{p(x)} \right)$$
  - Though it is not strictly a metric, KL divergence is indicative of how distant the distribution q is from the target distribution p

SyPy Security & Privacy Research Group

- KL divergence
  - "In other words, the KL divergence is the average number of extra bits needed to encode the data, due to the fact that we used distribution *q* to encode the data instead of the true distribution *p*." Page 58, Machine Learning: A Probabilistic Perspective, 2012.
  - $$D_{\mathrm{KL}}(p \parallel q) = H(p, q) - H(p) = -\sum_{x \in \mathcal{X}} p(x) \ \log_2 \left(\frac{q(x)}{p(x)} \right)$$
  - Though it is not strictly a metric, KL divergence is indicative of how distant the distribution q is from the target distribution p
  - For us, minimizing KL divergence is the same as minimizing cross entropy since original distribution does not change

SyPy Security & Privacy Research Group

Which classifier is better?

```
computed        | targets              | correct?
-----------------------------------------------
0.3  0.3  0.4   | 0  0  1              | yes
0.3  0.4  0.3   | 0  1  0              | yes
0.1  0.2  0.7   | 1  0  0              | no
```

```
computed        | targets              | correct?
-----------------------------------------------
0.1  0.2  0.7   | 0  0  1              | yes
0.1  0.7  0.2   | 0  1  0              | yes
0.3  0.4  0.3   | 1  0  0              | no
```

Which classifier is better?

```
computed        | targets              | correct?
-----------------------------------------------
0.3  0.3  0.4   | 0  0  1              | yes
0.3  0.4  0.3   | 0  1  0              | yes
0.1  0.2  0.7   | 1  0  0              | no
```

```
computed        | targets              | correct?
-----------------------------------------------
0.1  0.2  0.7   | 0  0  1              | yes
0.1  0.7  0.2   | 0  1  0              | yes
0.3  0.4  0.3   | 1  0  0              | no
```

Classification accuracy is a crude measure of NN performance!

Which classifier is better?

```
computed        | targets              | correct?
-----------------------------------------------
0.3  0.3  0.4   | 0  0  1              | yes
0.3  0.4  0.3   | 0  1  0              | yes
0.1  0.2  0.7   | 1  0  0              | no
```

```
computed        | targets              | correct?
-----------------------------------------------
0.1  0.2  0.7   | 0  0  1              | yes
0.1  0.7  0.2   | 0  1  0              | yes
0.3  0.4  0.3   | 1  0  0              | no
```

Here, cross entropy provides a far better idea for total error, provides much more granularity

Loss Functions

SyPy Security & Privacy Research Group

- Now, we have defined the metric for error
- Next, we have to come up with an algorithm such that weights can be adjusted in order to decrease this error
- We can now rewrite this as an optimization problem

$$\theta^* = \arg\min_\theta \mathcal{L}(\theta)$$

Evaluating ML models

SyPy Security & Privacy Research Group

Suppose you think of building an ML model since you have an awesome idea to solve image classification.

How do you decide when to stop model training?

How do you evaluate the performance of the model?

Enter performance metrics… HOMEWORK!

Finally…

SyPy
Security & Privacy
Research Group

Let us rest here

Questions?

Resources

SyPy Security & Privacy Research Group

- Linear Algebra (for reference/supplementary reading):
  - "Linear Algebra" by 3Blue1Brown
  - Deep Learning Book- Linear Algebra
- A python notebook where you can play around with some of the things we covered: https://github.com/udlbook/udlbook/blob/main/Notebooks/Chap01/1_1_BackgroundMathematics.ipynb
- Cross-Entropy: https://machinelearningmastery.com/cross-entropy-for-machine-learning/
- 3Blue1Brown Neural Network Playlist: https://youtube.com/playlist?list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi&si=i2YikXZLc81Qa4wl